


# Documentación del módulo transacciones de eShop Core

Instituto Superior de Formación Técnica N. º 179 Dr. Carlos Pellegrini

Carrera: Tecnicatura Superior en Análisis de Sistemas

Materia: Algoritmos y Estructura de Datos III

Docente: Hansen Andrés

Ayudante de cátedra: Ríos Emmanuel

Tema: Desarrollo de aplicación con TAD

Estudiantes: Trujillo Sebastián y Sanoguera Agustin

## Funcionamiento e integración del módulo de transacciones

El módulo BACKEND/transacciones.py coordina el registro de las compras del sistema. Valida el cliente y los productos solicitados, prepara los detalles, confirma la operación mediante persistencia y actualiza las estructuras compartidas del inventario, el historial y la popularidad. La compra se almacena en SQLite para recuperarla después de reiniciar el servidor.

Este documento explica sus siete funciones y su relación con servidor.py, persistencia.py, inventario.py, recomendaciones.py, esperas.py, el TDA Pila y el portal del cliente. Distingue el flujo de compra disponible en el sitio de una función de reversión que permanece en el código y no está expuesta por la API actual.

## Organización de los datos de una compra

Una compra preparada es un diccionario con id_cliente, fecha, productos y total. productos es una lista de detalles; cada detalle contiene id_producto, nombre, cantidad y precio_unitario. Cuando SQLite confirma el registro, se agrega id_transaccion, que identifica la compra en la base y en el historial.

La fecha se genera en UTC con precisión de segundos y formato ISO 8601. El total se calcula a partir de los precios del inventario y las cantidades agrupadas. Los precios y el total no se aceptan como valores elegidos por el navegador.

La pila almacena compras de todos los clientes del proceso. La consulta personal del servidor filtra posteriormente por id_cliente. La información persistente se organiza en transacciones para la cabecera y detalle_transaccion para los productos; los registros también se relacionan con clientes y productos.

## Uso del TDA Pila y política LIFO

El módulo utiliza Pila, apilar, desapilar, en_cima y pila_vacia, definidos en tad_pila.py. Las compras se incorporan por la cima. La política LIFO significa que la última incorporada es la primera accesible desde esa cima.

Por ejemplo, si se registran A, B y C en ese orden, la cima contiene C y debajo quedan B y A. Al consultar el historial se devuelve C, B, A. El módulo realiza las operaciones con las primitivas del TDA y no recorre ni modifica directamente los enlaces de sus nodos.

Apilar y desapilar son operaciones constantes respecto de la cantidad de compras, pero copiar una compra requiere recorrer sus datos. Por eso el costo de una consulta que utiliza deepcopy también depende de sus detalles.

## Funcionamiento de las funciones

### Validación de identificadores y cantidades

validar_entero_positivo(valor, campo) exige type(valor) is int y un valor entre uno y MAX_ENTERO_SQLITE, cuyo límite es 9223372036854775807. La comparación estricta rechaza booleanos, aunque Python los relacione con los enteros. También rechaza decimales, texto, cero, negativos y valores fuera del rango admitido.

La función no modifica datos ni devuelve un registro. Si la entrada es incorrecta, lanza ValueError con el nombre del campo. Se usa para el ID del cliente, los IDs de productos, las cantidades individuales y las cantidades acumuladas.

### Preparación de la compra

preparar_compra(inventario, id_cliente, productos_solicitados) comprueba el identificador del cliente y consulta su existencia en persistencia. Después exige una lista

no vacía. Cada elemento debe ser un diccionario con exactamente id_producto y cantidad; se rechazan campos faltantes o adicionales.

La función agrupa los productos repetidos en un diccionario de cantidades. Si un mismo producto aparece con dos y tres unidades, la cantidad solicitada final es cinco. La validación de stock se realiza contra ese total para evitar que varias líneas individualmente válidas excedan la disponibilidad conjunta.

Por cada producto agrupado consulta el inventario. Si no existe o no tiene unidades suficientes, lanza ValueError. Si existe, toma su nombre y su precio actual y forma el detalle. El total se acumula con Decimal a partir de la representación textual del precio, se convierte a float y se verifica con isfinite antes de devolver la compra preparada.

Esta etapa no inserta compras, no descuenta stock, no apila y no cambia la popularidad. Construye los datos que serán confirmados después. La comprobación inicial del inventario se complementa con otra verificación en SQLite durante el guardado.

**Consulta de la última transacción**

consultar_ultima_transaccion(pila) verifica si la pila está vacía. Si no hay compras, lanza ValueError. En caso contrario obtiene en_cima y devuelve una copia profunda mediante deepcopy, sin retirar el elemento.

La copia incluye la lista de productos y los diccionarios interiores. Así, modificar el resultado de la consulta no altera accidentalmente la compra conservada en la pila. La consulta se refiere a la compra global más reciente, no a la última de un cliente particular.

**Reconstrucción de la pila**

cargar_pila() crea una Pila vacía y solicita persistencia.leer_transacciones(). Esa lectura entrega las compras por id_transaccion ascendente. La función apila cada una en ese orden y devuelve el estado reconstruido.

Al apilar primero la más antigua y al final la más reciente, la última queda en la cima. Si no hay registros se devuelve una pila vacía. Esta carga no genera nuevas compras ni vuelve a descontar las existencias: recupera operaciones que ya estaban guardadas.

**Registro de una transacción**

registrar_transaccion(inventario, pila, popularidad, id_cliente, productos) coordina la compra completa. Primero llama a preparar_compra. Luego envía el resultado a persistencia.guardar_compra y agrega a la compra el id_transaccion devuelto.

Solo después de confirmar SQLite obtiene los IDs de los productos y llama a sincronizar_productos para actualizar el inventario desde la base. A continuación apila la compra y llama a recomendaciones.registrar_compra para sumar las unidades vendidas a la popularidad. Finalmente devuelve una copia profunda de la compra.

El orden evita cambiar deliberadamente las estructuras en memoria antes de que termine el guardado. La función presupone que el llamador mantiene el bloqueo compartido durante toda la operación; no crea ni adquiere su propio Lock.

**Consulta del historial sin perder la pila**

consultar_historial(pila) crea una pila auxiliar y una lista de resultados. Mientras la pila original tiene elementos, desapila una compra, la apila en la auxiliar y agrega una

copia profunda a la lista. Como comienza por la cima, el resultado queda ordenado desde la más reciente hacia la más antigua.

En el bloque finally vuelve a pasar todas las compras de la auxiliar a la original. Al terminar, se restituyen la secuencia y la cima que existían al comenzar. Con A, B y C, primero obtiene C, B, A y luego reconstruye la pila para que C vuelva a quedar arriba.

Si no hay compras devuelve una lista vacía. El recorrido requiere visitar todas las compras y copiar sus detalles. También necesita una pila auxiliar proporcional a la cantidad de transacciones. Como la consulta modifica temporalmente el estado compartido, el servidor la realiza bajo el mismo bloqueo que las compras.

**Reversión interna de la última compra**

deshacer_transaccion(inventario, colas, pila, popularidad, id_transaccion) consulta la cima y comprueba que su ID coincida con el solicitado. Antes de modificar datos también verifica que los conteos de popularidad permitan restar las cantidades de esa compra.

Luego llama a persistencia.revertir_ultima_compra, que verifica nuevamente cuál es la última compra guardada. Esa operación restituye stock y elimina cabecera y detalles en una misma transacción. Si el stock de un producto pasa de cero a positivo y existe una espera, también conserva un aviso y elimina la primera espera.

Tras confirmar la base, deshacer_transaccion sincroniza los productos, llama a confirmar_atencion por cada espera atendida, desapila la compra y resta sus unidades del árbol mediante recomendaciones.revertir_compra. Devuelve un diccionario con transaccion y notificaciones.

La reversión es global y solo permite la cima, no cualquier compra del historial. Su presencia responde al código interno conservado: servidor.py no importa

deshacer_transaccion ni ofrece una ruta para invocarla. El portal actual permite comprar y consultar las compras, sin una opción para deshacerlas.

## Relación con los demás módulos

### Servidor y sesión del cliente

servidor.py carga la pila una sola vez al iniciar y comparte ese estado entre sus manejadores. En POST /api/compras obtiene al cliente desde la sesión indicada por X-Sesion y exige un objeto JSON con la clave productos. El ID del cliente se toma de la sesión, no de un valor elegido dentro del carrito.

Bajo el Lock compartido llama a registrar_transaccion con inventario, pila, popularidad, cliente y productos. Si la operación termina correctamente responde con estado 201 y los datos de la compra. Los problemas de sesión producen 401; las validaciones y desbordamientos controlados, 400; y los errores SQLite, 500.

Para GET /api/mis-compras utiliza consultar_historial bajo el bloqueo y filtra las compras cuyo id_cliente coincide con el cliente actual. El historial de la pila es global, pero la respuesta personal incluye únicamente sus compras.

### Persistencia y confirmación en SQLite

persistencia.py proporciona consultar_cliente, leer_transacciones, guardar_compra y revertir_ultima_compra. transacciones.py no escribe SQL directamente. La persistencia agrupa las escrituras duraderas y devuelve los resultados con los que se actualizan las estructuras.

guardar_compra utiliza BEGIN IMMEDIATE y vuelve a comprobar cliente, productos, stock y precios actuales. Recalcula el total y guarda cabecera, detalles y descuento de existencias dentro de la misma transacción. Si una escritura falla, no se confirma solo una parte del carrito.

Al reconstruir el historial, leer_transacciones conserva cantidad y precio_unitario desde los detalles guardados y obtiene el nombre mediante una unión con productos. El nombre mostrado después de reiniciar puede reflejar un cambio posterior del catálogo; el precio unitario sigue siendo el registrado para la compra.

**Inventario y TDA Lista**

inventario.consultar_producto devuelve una copia del producto localizado en la lista enlazada. Esa consulta permite preparar cantidades, nombres y precios sin modificar el dato original. Después de guardar la compra, inventario.sincronizar_productos lee de SQLite los productos afectados y los reemplaza mediante las primitivas de lista.

La disminución de stock ocurre dentro de guardar_compra. Sincronizar no realiza otro descuento: refleja en memoria el valor confirmado en la base. El módulo de transacciones utiliza la interfaz del inventario y no administra directamente los enlaces de la lista.

**Recomendaciones y árbol BST**

recomendaciones.registrar_compra suma las cantidades compradas por producto. La popularidad representa unidades vendidas. Si una compra contiene tres unidades de un artículo, su conteo aumenta en tres, no en uno.

El BST utiliza claves (popularidad, id_producto). Cuando cambia el conteo, recomendaciones elimina la clave anterior e inserta la nueva. transacciones.py entrega los detalles de la compra y delega el manejo del árbol. Al reiniciar, recomendaciones reconstruye el estado con las sumas de detalle_transaccion consultadas mediante persistencia.

El stock disponible y la popularidad son datos diferentes: la compra reduce el primero y aumenta la segunda. Este flujo no usa un árbol de stock ni calcula el producto con menor disponibilidad.

**Esperas y TDA Cola**

Una compra normal no retira reservas de las colas ni asigna existencias a un cliente en espera. Si un producto se agota, las nuevas solicitudes de reserva se procesan por esperas.py a través de otra operación del portal.

La dependencia directa de transacciones.py con confirmar_atencion aparece en la reversión interna. Si la restitución de stock atiende una espera en SQLite, se retira su registro del frente de la cola para sincronizar la memoria. El flujo habitual de reposición desde inventario también coordina los avisos y las colas, pero corresponde a inventario.py.

**Portal de compras y carrito**

portal.js envía el carrito a POST /api/compras cuando el cliente selecciona comprar. El navegador transmite IDs y cantidades. Tras recibir una confirmación correcta vacía el carrito, muestra el número y total de la compra y actualiza los datos del portal.

Si ocurre un error, muestra el mensaje y recomienda consultar Tus compras antes de confirmar nuevamente. Esto importa si la operación se guardó pero la respuesta no llegó al navegador. Vaciar el carrito por decisión del usuario solo borra sus productos pendientes; no elimina compras persistidas ni revierte el stock.

**Flujo completo de una compra**

El recorrido comienza con el cliente identificado y los productos añadidos al carrito. El portal envía la solicitud y el servidor verifica su sesión y formato. Dentro del

bloqueo compartido, preparar_compra valida las entradas, agrupa los productos repetidos y obtiene disponibilidad y precios.

guardar_compra verifica otra vez los datos contra SQLite y confirma el registro con su detalle y el descuento de stock. registrar_transaccion recupera los productos actualizados, incorpora la compra a la pila y modifica la popularidad. Devuelve el resultado al servidor, que responde al navegador.

El portal presenta la confirmación y consulta nuevamente catálogo, recomendaciones y compras personales. En un inicio posterior, los registros guardados permiten reconstruir la lista, la pila y el árbol sin volver a ejecutar la venta.

**Ejemplo del intercambio entre estructuras**

Como ejemplo conceptual, un producto tiene diez unidades disponibles y una popularidad previa de cinco. El cliente solicita dos unidades a un precio de mil pesos cada una. La preparación genera un detalle con cantidad dos y total de dos mil pesos.

Al confirmar, SQLite guarda la compra y deja el stock en ocho. El inventario refleja esas ocho unidades; la compra queda en la cima de la pila; y recomendaciones cambia la popularidad de cinco a siete. La cola de espera conserva sus registros, porque una compra ordinaria no realiza una atención.

Este ejemplo describe el comportamiento del código y no crea datos de prueba en la base.

**Consistencia de los datos y límites de responsabilidad**

deepcopy se utiliza porque una compra incluye objetos anidados. Permite devolver consultas y resultados que el llamador puede modificar sin alterar la transacción conservada en la pila. No crea otra fila en SQLite ni un respaldo del archivo de base de datos.

El Lock coordina las estructuras compartidas dentro del proceso del servidor. La transacción SQLite protege el conjunto de escrituras en el archivo. No existe una única transacción que abarque también las modificaciones posteriores de lista, pila y árbol: si se produce un fallo después de confirmar la base, el código no revierte automáticamente toda la memoria.

La recuperación al iniciar vuelve a construir los estados desde SQLite. El módulo tampoco implementa una clave de idempotencia para reconocer una repetición del mismo pedido HTTP. Por eso una respuesta perdida no debe interpretarse automáticamente como una compra inexistente; consultar el historial ayuda a comprobar si quedó registrada.

Las responsabilidades quedan distribuidas entre la interfaz que recoge la intención de compra, el servidor que identifica al cliente y coordina solicitudes, transacciones.py que valida y organiza el flujo, persistencia.py que confirma los datos duraderos y los módulos que mantienen cada TDA. Esta documentación describe esa integración sin habilitar funciones nuevas ni modificar las reglas del sistema.